



NAAC
GRADE A+
Accredited University



CHANDIGARH
UNIVERSITY
Discover. Learn. Empower.

PROJECT REPORT

Contact Management System

Using C Programming Language

Subject: - Data Structures Lab
Subject Code: - 25CAP-152

SUBMITTED BY:

Name: Somya Gupta

Branch: UIC

Semester: 02

UID: 25BCD10029

Group: A, Section: BCD-1

SUBMITTED TO:

Name: Riya Mondal Ma'am

D.O. SUBMISSION: 21/04/2026

Designation: AP

Sign: _____

INDEX

Sr. No	Content	Page No
01	Title Page	01
02	Index	02
03	Introduction	03
04	Objectives	04
05	Data Description	04
06	Tools and Technologies	04 – 05
07	Complete Code with Explanation	05 – 09
08	Program Output	10 – 11
09	Conclusion	12
10	Learning Outcomes	12
11	GitHub Link https://github.com/Somya-ds/Contact-Management-System-Using-C	12

1. Introduction

In today's fast-paced digital world, managing contacts efficiently has become a necessity for both personal and professional purposes. Earlier, people relied on traditional methods such as diaries or address books to store contact details. However, these methods were prone to data loss, difficult to update, and inefficient for searching large amounts of information.

To overcome these limitations, digital solutions like Contact Management Systems have been developed. This project presents a **Contact Management System implemented in the C programming language**, designed to store, organize, and manage contact information efficiently.

The system is a **menu-driven console application** that allows users to perform operations such as adding new contacts, deleting existing contacts, displaying all contacts, sorting them alphabetically, and searching contacts using efficient algorithms. It also includes **duplicate prevention** and **binary search optimization**, making it more advanced than basic implementations.

This project highlights the importance of **data structures and algorithms** in solving real-world problems and demonstrates how efficient data handling improves system performance.

	Add Contacts	Dynamically add new contacts with name, phone & email
	Delete Contacts	Remove contacts safely with proper memory deallocation
	Binary Search	Fast $O(\log n)$ search after converting list to array
	Sort Contacts	Alphabetical ordering using linked-list insertion sort
	Display All	Traverse and print the full contact list at any time
	No Duplicates	Linear search duplicate check before every insertion

2. Objectives

The primary objectives of this project are:

- Design and implement a Contact Management System using C programming language
- Understand and apply linked list data structure for dynamic data storage
- Implement efficient insertion and deletion operations on linked lists
- Apply linear search for duplicate prevention and binary search for fast lookup
- Implement insertion sort algorithm directly on the linked list
- Prevent duplicate entries to maintain data integrity
- Build a clean menu-driven application for smooth user interaction
- Enhance problem-solving, debugging, and logical thinking skills

3. Data Description

Each contact is stored as a node in a singly linked list. The node is defined using the following C structure:

Field	Data Type	Description
name[50]	char array	Full name of the contact (max 50 characters)
phone[15]	char array	Mobile/phone number (max 15 characters)
email[50]	char array	Email address of the contact (max 50 characters)
next	struct Contact	Pointer to the next node — forms the linked list

4. Tools and Technologies

4.1 Programming Language

The entire project is implemented in the C Programming Language — chosen for its efficiency, direct memory control, and suitability for building data structures from scratch.

4.2 Libraries Used

Library	Purpose
<code>stdio.h</code>	Standard input/output — <code>printf()</code> , <code>scanf()</code>
<code>stdlib.h</code>	Dynamic memory management — <code>malloc()</code> , <code>free()</code> , <code>exit()</code>
<code>string.h</code>	String manipulation — <code>strcmp()</code> , <code>strcpy()</code> , <code>strlen()</code>

4.3 Compatible Platforms

- Turbo C++ (Classic DOS-based compiler)
- CodeBlocks IDE (Cross-platform C/C++ IDE)
- Visual Studio Code with GCC / MinGW compiler
- Any standard C99 or C11 compliant compiler.

5. Complete Code with Explanation

The following section presents the complete C source code split into labelled, colour-coded segments. Each segment is preceded by an explanation of its purpose and the algorithmic logic applied.

■ Keywords

■ Built-ins

■ Comments

■ Strings

■ Numbers

■ Identifiers

01 Headers & Structure Definition

The program begins by including standard libraries and defining the `Contact` structure with a self-referential pointer to build the linked list.

```
//Contact Management System

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure
struct Contact {
    char name[50];
    char phone[15];
    char email[50];
    struct Contact* next;
};

struct Contact* head = NULL;
```

02 Duplicate Check — isDuplicate()

Before inserting any new contact, this function traverses the linked list using linear search to detect if a contact with the same name already exists, ensuring data integrity.

```
// Check Duplicate
int isDuplicate(char name[]) {
    struct Contact* temp = head;
    while (temp != NULL) {
        if (strcmp(temp->name, name) == 0)
            return 1;
        temp = temp->next;
    }
    return 0;
}
```

03 Add Contact — addContact()

Dynamically allocates memory for a new node, reads user input, checks for duplicates, and inserts the node at the head of the linked list for O(1) insertion.

```
// Add Contact
void addContact() {
    struct Contact* newNode = (struct Contact*)malloc(sizeof(struct Contact));

    printf("Enter Name: ");
    scanf(" %[^\\n]", newNode->name);

    if (isDuplicate(newNode->name)) {
        printf("Contact already exists!\\n");
        free(newNode);
        return;
    }

    printf("Enter Phone: ");
    scanf("%s", newNode->phone);

    printf("Enter Email: ");
    scanf("%s", newNode->email);

    newNode->next = head;
    head = newNode;

    printf("Contact Added Successfully!\\n");
}
```

04 Display Contacts — displayContacts()

Traverses the linked list from head to tail and prints all contact details. Handles the empty list case gracefully.

```
// Display Contacts
void displayContacts() {
```

```
struct Contact* temp = head;

if (temp == NULL) {
    printf("No contacts available!\n");
    return;
}

printf("\n--- Contact List ---\n");
while (temp != NULL) {
    printf("Name: %s | Phone: %s | Email: %s\n",
        temp->name, temp->phone, temp->email);
    temp = temp->next;
}
}
```

05 Delete Contact — deleteContact()

Searches for a contact by name, adjusts the previous node's next pointer to bypass the target node, then frees the allocated memory to prevent memory leaks.

```
// Delete Contact
void deleteContact() {
    char name[50];
    printf("Enter name to delete: ");
    scanf(" %s", name);

    struct Contact *temp = head, *prev = NULL;

    while (temp != NULL && strcmp(temp->name, name) != 0) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Contact not found!\n");
        return;
    }

    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;

    free(temp);
    printf("Contact Deleted Successfully!\n");
}
```

06 Sort Contacts — sortContacts()

Implements insertion sort directly on the linked list — no array conversion needed. Each node is removed from the original list and re-inserted into its correct alphabetical position in a new sorted list.

```
// Sort Contacts (Insertion Sort)
void sortContacts() {
```

```
struct Contact *sorted = NULL;
struct Contact* current = head;

while (current != NULL) {
    struct Contact* next = current->next;

    if (sorted == NULL || strcmp(sorted->name, current->name) >= 0) {
        current->next = sorted;
        sorted = current;
    } else {
        struct Contact* temp = sorted;
        while (temp->next != NULL &&
            strcmp(temp->next->name, current->name) < 0) {
            temp = temp->next;
        }
        current->next = temp->next;
        temp->next = current;
    }
    current = next;
}
head = sorted;
printf("Contacts Sorted Successfully!\n");
}
```

07 List to Array & Binary Search

Since binary search requires indexed access, `listToArray()` copies linked list node pointers into an array. `binarySearchContact()` then performs $O(\log n)$ binary search on the sorted array.

```
// Convert Linked List to Array
int listToArray(struct Contact* arr[]) {
    struct Contact* temp = head;
    int i = 0;
    while (temp != NULL) {
        arr[i++] = temp;
        temp = temp->next;
    }
    return i;
}

// Binary Search
void binarySearchContact() {
    struct Contact* arr[100];
    int n = listToArray(arr);

    if (n == 0) {
        printf("No contacts available!\n");
        return;
    }

    char key[50];
    printf("Enter name to search (sorted list required): ");
    scanf(" %s", key);

    int low = 0, high = n - 1;

    while (low <= high) {
```

```
int mid = (low + high) / 2;
int cmp = strcmp(arr[mid]->name, key);

if (cmp == 0) {
    printf("Found: %s | %s | %s\n",
        arr[mid]->name, arr[mid]->phone, arr[mid]->email);
    return;
} else if (cmp < 0) {
    low = mid + 1;
} else {
    high = mid - 1;
}

printf("Contact not found!\n");}
```

08 Main Function — Program Entry Point

The `main()` function runs an infinite loop presenting a menu to the user. A switch statement routes each selection to the appropriate function, with option 6 gracefully exiting the program.

```
// Main
int main() {
    int choice;

    while (1) {
        printf("\n--- Contact Management System ---\n");
        printf("1. Add Contact\n");
        printf("2. Display Contacts\n");
        printf("3. Delete Contact\n");
        printf("4. Sort Contacts\n");
        printf("5. Binary Search Contact\n");
        printf("6. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: addContact(); break;
            case 2: displayContacts(); break;
            case 3: deleteContact(); break;
            case 4: sortContacts(); break;
            case 5: binarySearchContact(); break;
            case 6: exit(0);
            default: printf("Invalid choice!\n");
        }
    }
}
```

6. Program Output

Output

```
--- Contact Management System ---
```

1. Add Contact
2. Display Contacts
3. Delete Contact
4. Sort Contacts
5. Binary Search Contact
6. Exit

```
Enter choice: 1
```

```
Enter Name: Somya
```

```
Enter Phone: 7973704290
```

```
Enter Email: somya2412@gmail.com
```

```
Contact Added Successfully!
```

```
--- Contact Management System ---
```

1. Add Contact
2. Display Contacts
3. Delete Contact
4. Sort Contacts
5. Binary Search Contact
6. Exit

```
Enter choice: 1
```

```
Enter Name: Mrithul
```

```
Enter Phone: 8765392100
```

```
Enter Email: mrithul2004@gmail.com
```

```
Contact Added Successfully!
```

```
--- Contact Management System ---
```

1. Add Contact
2. Display Contacts
3. Delete Contact
4. Sort Contacts
5. Binary Search Contact
6. Exit

```
Enter choice: 2
```

```
--- Contact List ---
```

```
Name: Mrithul | Phone: 8765392100 | Email: mrithul2004@gmail.com
```

```
Name: Somya | Phone: 7973704290 | Email: somya2412@gmail.com
```

```
--- Contact Management System ---
1. Add Contact
2. Display Contacts
3. Delete Contact
4. Sort Contacts
5. Binary Search Contact
6. Exit
Enter choice: 4
Contacts Sorted Successfully!

--- Contact Management System ---
1. Add Contact
2. Display Contacts
3. Delete Contact
4. Sort Contacts
5. Binary Search Contact
6. Exit
Enter choice: 5
Enter name to search (sorted list required): Mrithul
Found: Mrithul | 8765392100 | mrithul2004@gmail.com
```

```
--- Contact Management System ---
1. Add Contact
2. Display Contacts
3. Delete Contact
4. Sort Contacts
5. Binary Search Contact
6. Exit
Enter choice: 6

=== Code Execution Successful ===
```

7. Conclusion

The Contact Management System developed in C successfully demonstrates the use of fundamental data structures and algorithms in solving real-world problems. By using a singly linked list, the system efficiently handles dynamic data without memory wastage or predefined size limitations.

The implementation of insertion, deletion, sorting, and searching operations ensures that the system is both functional and efficient. Features such as duplicate prevention and binary search significantly enhance the system's reliability and performance, demonstrating the importance of choosing the right data structure and algorithm for a given problem.

8. Learning Outcomes

Through the development and implementation of this project, the following key concepts and skills were acquired:

1. In-depth understanding of **linked list data structure** and its real-world applicability
2. Practical implementation of **dynamic memory allocation** using malloc() and free()
3. Application of **linear and binary search algorithms**
4. Mastery of **insertion sort** applied directly on linked list nodes
5. Handling real-world problems by designing structured, modular C programs
6. Improved debugging techniques and logical thinking skills
7. Writing clean, well-commented, and maintainable C code

9. GitHub Link

<https://github.com/Somya-ds/Contact-Management-System-Using-C/tree/main>
